

Day2 실습 실행결과

광주_3반_임해안

실습 4 – Pandas 2.x 데이터 정제

python Python_data_0720_Day2/practice4/pr4.py

```
(.venv) limhaean@imhaean-ui-MacBookPro skala_python % python Python_data_0720_Day2/practice4/pr4.py | tee /tmp/pr4_result.txt

-- STEP 0: 원본 데이터 진단 --
데이터 파일: /Users/limhaean/workspace/skala_python/data/sales_raw.csv
행·열 개수: (5000, 7)

[컬럼 타입·결측·메모리 정보]
<class 'pandas.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  -
0   order_id    5000 non-null   str
1   order_date  5000 non-null   str
2   region      4694 non-null   str
3   category    5000 non-null   str
4   quantity    5000 non-null   int64
5   unit_price  4790 non-null   float64
6   discount    5000 non-null   float64
dtypes: float64(2), int64(1), str(4)
memory usage: 429.6 KB

[수치형 기술통계]
      quantity    unit_price    discount
count  5000.000000    4790.000000  5000.000000
mean    18.561200   151030.201879    0.049370
std    134.098936   88652.988253    0.057365
min      1.000000  -299111.000000    0.000000
25%      3.000000   77044.000000    0.000000
50%      6.000000  151162.500000    0.000000
75%      8.000000  227002.500000    0.100000
max    1995.000000  299608.000000    0.150000

[컬럼별 결측치 수]
order_id      0
order_date    0
region       306
category      0
quantity      0
unit_price    210
discount      0
dtype: int64
```

```
[앞 5건]
  order_id  order_date  region  category  quantity  unit_price  discount
0  ORD-000000  2025-04-07  Busan    Beauty      6    12609.0    0.00
1  ORD-000001  2025-04-08  Busan    Home      1    262252.0    0.05
2  ORD-000002  2025-03-01  Gwangju  Beauty      3    34466.0    0.15
3  ORD-000003  2025-01-11  Seoul  Electronics  3    127645.0    0.00
4  ORD-000004  2025-04-16  Busan    Beauty      3    46037.0    0.00
```

--- STEP 1: 타입 정규화 ---

```
변환 전          변환 후
order_id         str          str
order_date       str  datetime64[us]
region           str          str
category         str          category
quantity         int64        int64
unit_price       float64      float64
discount         float64      float64
```

[타입 변환으로 추가된 결측치 수]

```
order_id      0
order_date    0
region        0
category      0
quantity      0
unit_price    0
discount      0
dtype: int64
```

--- STEP 2: 가격 중앙값·지역 최빈값 결측치 처리 ---

```
처리 전  처리 후  처리 건수
order_id      0      0
order_date    0      0
region        306    0  306
category      0      0
quantity      0      0
unit_price    210    0  210
discount      0      0
가격 업무규칙 위반 (0 이하): 29건 → 0건
지역 결측 대체값 (최빈값): Gwangju
행 수 확인: 5,000 → 5,000
```

--- STEP 3: IQR 이상치 원저라이징 ---

```
IQR 하한  IQR 상한  이상치 수  처리 전 최솟값  처리 전 최댓값  처리 후 최솟값  처리 후 최댓값
컬럼
unit_price -127851.38  435077.62      0    5049.0  299608.0    5049.0  299608.0
quantity   -4.50    15.50     53     1.0  1995.0     1.0    15.0
행 수 확인: 5,000 → 5,000
```

--- STEP 4: 카테고리별 매출 요약 ---

```
건수  평균가  중앙값  총매출
category
Beauty    1012  156526.9  159479.0  824667442.2
Electronics  995  150542.3  143345.0  803187797.2
Fashion    967  155762.0  153534.0  811835494.0
Food       1049  150797.8  150472.0  838002283.5
Home       977  150478.7  153996.5  805336411.2
```

--- STEP 5: 카테고리·지역별 매출 교차표 ---

```
region      Busan      Daegu      Gwangju      Incheon      Seoul
category
Beauty    146827187.8  136293066.4  198633726.5  175404563.5  167508897.9
Electronics  151041213.2  155643225.4  217545958.0  143833881.2  135123519.5
Fashion    158626728.4  165390262.4  216390216.4  147362268.2  124066018.6
Food       141443927.1  163241573.2  215077602.4  153483823.0  164755357.8
Home       149369673.3  154881415.6  202180682.1  148525630.8  150379009.3
```

지역 결측으로 교차표에서 제외된 행: 0건

--- STEP 6: 카테고리 기준정보 결합 ---

행 수 확인: 5,000 → 5,000

[결합 상태]

```
_merge
both      5000
left_only  0
right_only 0
```

[결합 결과 앞 5건]

```
order_id  category  category_ko
ORD-000000  Beauty      뷰티
ORD-000001  Home        생활용품
ORD-000002  Beauty      뷰티
ORD-000003  Electronics  전자제품
ORD-000004  Beauty      뷰티
```

--- STEP 7: Copy-on-Write와 .loc 안전 수정 ---

Pandas 버전: 3.0.3

Copy-on-Write: 원본 보호 확인

unit_price > 100,000 고가 플래그: 3,488건

판매 원천 데이터 5,000건의 타입·결측치·기술통계를 진단하고 숫자·날짜·범주형 컬럼을 분석에 적합한 자료형으로 정규화하였다.

pytest -v Python_data_0720_Day2/practice4/test_pr4.py

```
(.venv) linhaean@linhaean-ui-MacBookPro skala_python % pytest -v Python_data_0720_Day2/practice4/test_pr4.py
===== test session starts =====
platform darwin -- Python 3.11.15, pytest-8.4.2, pluggy-1.6.0 -- /Users/linhaean/workspace/skala_python/.venv/bin/python
cachedir: .pytest_cache
rootdir: /Users/linhaean/workspace/skala_python
collected 6 items

Python_data_0720_Day2/practice4/test_pr4.py::test_normalize_types_converts_invalid_values_to_missing PASSED [ 16%]
Python_data_0720_Day2/practice4/test_pr4.py::test_fill_missing_price_uses_each_category_median PASSED [ 33%]
Python_data_0720_Day2/practice4/test_pr4.py::test_invalid_prices_are_replaced_with_category_median PASSED [ 50%]
Python_data_0720_Day2/practice4/test_pr4.py::test_fill_missing_region_uses_mode PASSED [ 66%]
Python_data_0720_Day2/practice4/test_pr4.py::test_winsorize_outliers_clips_values_and_preserves_rows PASSED [ 83%]
Python_data_0720_Day2/practice4/test_pr4.py::test_cleaning_pipeline_keeps_all_input_rows PASSED [100%]

===== 6 passed in 0.39s =====
```

타입 정규화, 그룹별 중앙값 대체, 비정상 가격 처리, 지역 최빈값 대체, IQR
원저라이징과 전체 행 보존 규칙을 함수로 분리하고 pytest로 자동 검증하였다. 총 6개의
테스트가 모두 통과하였다.

실습 5 - Polars· DuckDB 성능 비교

python Python_data_0720_Day2/practice5/pr5.py

```
(.venv) limhaean@limhaean-ui-MacBookPro skala_python % python Python_data_0720_Day2/practice5/pr5.py

-- STEP 0: 공통 비교 질의 정의 --
데이터 파일: /Users/limhaean/workspace/skala_python/data/events_large.csv
전체 데이터 행 수: 1,000,000
컬럼: ['event_id', 'user_id', 'event_type', 'ts', 'amount']
공통 질의: amount가 0보다 큰 행만 선택하고, event_type별로 묶어 건수(cnt)와 평균 금액(avg)을 계산한 뒤 건수 내림차순으로 정렬한다.

[앞 5건]
1: {'event_id': '0', 'user_id': '33864', 'event_type': 'view', 'ts': '2025-03-29 12:01:33', 'amount': '0'}
2: {'event_id': '1', 'user_id': '41673', 'event_type': 'view', 'ts': '2025-03-20 23:54:27', 'amount': '0'}
3: {'event_id': '2', 'user_id': '46010', 'event_type': 'view', 'ts': '2025-01-01 17:18:23', 'amount': '0'}
4: {'event_id': '3', 'user_id': '23520', 'event_type': 'view', 'ts': '2025-02-24 20:51:01', 'amount': '0'}
5: {'event_id': '4', 'user_id': '27346', 'event_type': 'view', 'ts': '2025-03-27 11:33:32', 'amount': '0'}

-- STEP 1: Pandas 기준선 --
event_type  cnt      avg
purchase 60062 251706.059372
refund 19844 250129.096604
Pandas 실행 시간: 307.52 ms

-- STEP 2: Polars Lazy API --
shape: (2, 3)


| event_type | cnt   | avg           |
|------------|-------|---------------|
| str        | u32   | f64           |
| purchase   | 60062 | 251706.059372 |
| refund     | 19844 | 250129.096604 |


Polars 실행 시간: 19.06 ms

-- STEP 3: DuckDB SQL --
event_type  cnt      avg
purchase 60062 251706.059372
refund 19844 250129.096604
DuckDB 실행 시간: 57.78 ms

-- STEP 4: 세 엔진 결과 일치 검증 --
세 엔진 결과 일치: assert_frame_equal 통과

-- STEP 5: 세 엔진 성능 비교 --
동일 반복 횟수: 3회


| 엔진     | 반복별 시간 (ms)            | 중앙값 (ms) | Pandas 대비 |
|--------|------------------------|----------|-----------|
| Polars | 14.89, 14.43, 13.35    | 14.43    | 18.9x     |
| DuckDB | 53.84, 53.99, 51.75    | 53.84    | 5.1x      |
| Pandas | 286.85, 272.55, 271.36 | 272.55   | 1.0x      |


```

100만 건의 이벤트 데이터에 동일한 필터·그룹 집계 질의를 Pandas, Polars Lazy API와 DuckDB SQL로 각각 실행하였다.

정렬·타입 차이·부동소수점 허용오차를 반영하여 세 엔진의 결과가 동일함을 검증하였다. 각 엔진을 3회 반복 측정한 중앙값 기준으로 Polars, DuckDB, Pandas 순의 실행 속도를 확인하였다.

종합실습 2 - EDA + 통계 + ML 파이프라인

python Python_data_0720_Day2/Total2/analysis.py

```
(.venv) Limhaean@Limhaean-ui-MacBookPro skala_python % python Python_data_0720_Day2/Total2/analysis.py

-- STEP 0: Polars 기초 EDA --
데이터 파일: /Users/Limhaean/workspace/skala_python/data/telco_churn.csv
행·열 개수: (7000, 10)
컬럼: ['customer_id', 'gender', 'senior', 'tenure_months', 'monthly_charges', 'total_charges', 'contract', 'payment_method', 'num_services', 'churn']

[할 5건]
shape: (5, 10)

```

customer_id	gender	senior	tenure_months		contract	payment_method	num_services	churn
str	str	i64	i64		str	str	i64	i64
CUST-00000	Male	0	47	--	Two year	Credit card	7	0
CUST-00001	Male	0	23	--	One year	Bank transfer	4	0
CUST-00002	Female	0	60	--	Two year	Bank transfer	8	0
CUST-00003	Male	0	8	--	Month-to-month	Electronic check	3	1
CUST-00004	Male	0	32	--	One year	Credit card	4	1

```

[기술통계]
shape: (9, 11)

```

statistic	customer_id	gender	senior		contract	payment_method	num_services	churn
str	str	str	f64		str	str	f64	f64
count	7000	7000	7000.0	--	7000	7000	7000.0	7000.0
null_count	0	0	0.0	--	0	0	0.0	0.0
mean	null	null	0.248714	--	null	null	4.467143	0.240714
std	null	null	0.432299	--	null	null	2.295826	0.427548
min	CUST-00000	Female	0.0	--	Month-to-month	Bank transfer	1.0	0.0
25%	null	null	0.0	--	null	null	2.0	0.0
50%	null	null	0.0	--	null	null	4.0	0.0
75%	null	null	0.0	--	null	null	6.0	0.0
max	CUST-06999	Male	1.0	--	Two year	Mailed check	8.0	1.0

```
[이탈 여부별 고객 수와 비율]
shape: (2, 3)
```

churn	count	ratio_pct
i64	u32	f64
0	5315	75.93
1	1685	24.07

소수 클래스 비율: 24.07%

평가 지표: 클래스 불균형을 고려해 ROC-AUC를 사용합니다.

```
-- STEP 1: 이탈 여부별 고객 특성 비교 --
shape: (2, 4)
```

churn	avg_monthly_charges	avg_tenure_months	count
i64	f64	f64	u32
0	67.26	36.62	5315
1	75.11	34.02	1685

이탈 고객의 평균 월요금 차이: +7.85

이탈 고객의 평균 가입기간 차이: -2.60개월

가설: 월요금과 이탈 여부가 연관되어 있는지 통계 검정이 필요합니다.

```
-- STEP 2: Plotly HTML 시각화 --
```

HTML 리포트: /Users/limhaean/workspace/skala_python/Python_data_0720_Day2/Total2/output/churn_charges.html

파일 크기: 4,924.5 KB

```
-- STEP 3: 통계 검정 --
```

Welch t-검정: t=9.3889, p=1.23e-20

월요금 평균 차이는 통계적으로 유의합니다.

카이제곱 검정: chi2=321.7993, dof=2, p=1.32e-70

계약 유형과 이탈 여부의 연관성은 통계적으로 유의합니다.

해석 주의: 통계적 연관성이 인과관계를 의미하지는 않습니다.

```

-- STEP 4: 모델 데이터 준비와 전처리 전략 --
특성 행·열 개수: (7000, 8)
수치형 특성: ['senior', 'tenure_months', 'monthly_charges', 'total_charges', 'num_services']
범주형 특성: ['gender', 'contract', 'payment_method']

[특성별 결측치 수]
total_charges    162
타깃 결측치 수: 0
전처리 전략: 수치형은 중앙값 대체·표준화, 범주형은 One-Hot 인코딩
누수 방지: 전처리는 train/test 분리 후 Pipeline 내부에서 학습합니다.

-- STEP 5: ColumnTransformer 구성 --
ColumnTransformer(transformers=[('numeric',
    Pipeline(steps=[('imputer',
        SimpleImputer(strategy='median')),
        ('scaler', StandardScaler())]),
    ['senior', 'tenure_months', 'monthly_charges',
     'total_charges', 'num_services']),
    ('categorical',
    Pipeline(steps=[('imputer',
        SimpleImputer(strategy='most_frequent')),
        ('onehot',
        OneHotEncoder(handle_unknown='ignore'))]),
    ['gender', 'contract', 'payment_method'])])

현재 상태: 구성 완료, 아직 fit하지 않음
학습 시점: 다음 단계에서 train 데이터에 Pipeline 전체를 fit

-- STEP 6: RandomForest Pipeline 학습 --
train 크기: (5600, 8), 이탈 비율: 24.07%
test 크기: (1400, 8), 이탈 비율: 24.07%
Pipeline 단계: ['preprocessor', 'model']
교차검증 최적 파라미터: {'model__max_depth': 6, 'model__min_samples_leaf': 20}
train 5-fold 평균 ROC-AUC: 0.679
학습 완료: 모델 선택과 전처리는 train 데이터 안에서만 수행했습니다.

-- STEP 7: 모델 평가 및 저장 --
ROC-AUC: 0.673
train OOF 선택 임계값: 0.269 (정밀도=0.365, 재현율=0.600)

[분류 리포트]

```

	precision	recall	f1-score	support
전류	0.832	0.667	0.740	1063
이탈	0.354	0.576	0.438	337
accuracy			0.645	1400
macro avg	0.593	0.621	0.589	1400
weighted avg	0.717	0.645	0.668	1400

```

모델 파일: /Users/limhaean/workspace/skala_python/Python_data_0720_Day2/Total2/output/churn_model.joblib
파일 크기: 1,338.5 KB
재로딩 예측 일치: 확인

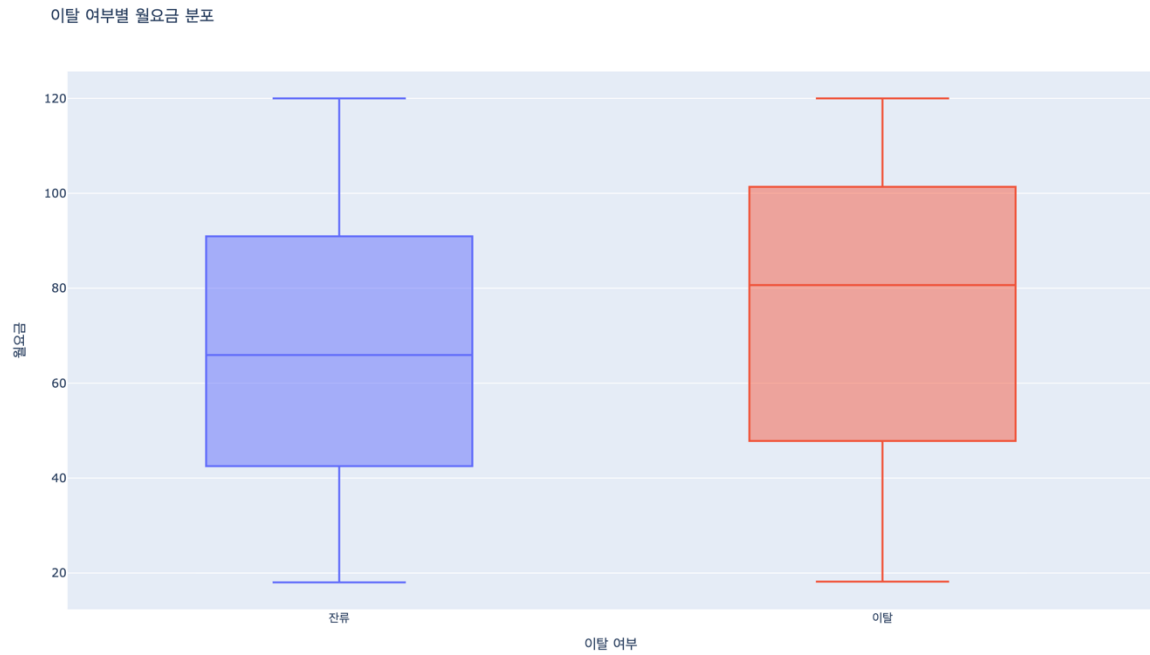
```

Polars로 7,000명의 고객 데이터를 탐색한 결과 이탈 고객 비율이 24.07%인 불균형 데이터임을 확인하고 ROC-AUC를 주 평가 지표로 선정하였다.

월요금 차이와 계약 유형의 연관성을 통계적으로 검정한 결과 모두 유의하였다. 이는 연관성을 의미하며 인과관계를 의미하지는 않는다. 전처리는 데이터 누수를 방지하기 위해 Pipeline 내부에 구성하였다.

계층 분할과 train 내부 교차검증으로 RandomForest 파라미터를 선택하였다. Test ROC-AUC 0.673을 달성했으며, 이탈 재현율 개선을 위해 train OOF 임계값을 적용하였다. Pipeline과 임계값을 저장한 후 재로딩 예측도 일치함을 확인하였다.

Python_data_0720_Day2/Total2/output/churn_charges.html



Plotly 박스플롯을 이용해 이탈 여부별 월요금 분포와 이상치를 시각적으로 비교하였다.

ls -lh Python_data_0720_Day2/Total2/output

```
(.venv) limhaean@imhaean-ui-MacBookPro skala_python % ls -lh Python_data_0720_Day2/Total2/output
total 13960
-rw-r--r--@ 1 limhaean  staff   4.8M Jul 21 20:40 churn_charges.html
-rw-r--r--@ 1 limhaean  staff   1.3M Jul 21 20:40 churn_model.joblib
(.venv) limhaean@imhaean-ui-MacBookPro skala_python %
```

추가 부분 - 모델 성능 개선

4. 개선 결과			
항목	기존 모델(0.5)	규제 모델(0.5)	최종 모델(0.269)
모델 선택 방식	기본값	train 5-fold 교차검증	동일
<code>max_depth</code>	제한 없음	6	6
<code>min_samples_leaf</code>	1	20	20
train 성능	ROC-AUC 1.000	CV ROC-AUC 0.679	동일
test ROC-AUC	0.635	0.673	0.673
test 정확도	0.742	0.752	0.645
이탈 정밀도	0.414	0.250	0.354
이탈 재현율	0.172	0.015	0.576
이탈 F1-score	0.243	0.028	0.438
저장 모델 크기	약 40MB	약 1.3MB	약 1.3MB

```

-- STEP 6: RandomForest Pipeline 학습 --
train 크기: (5600, 8), 이탈 비율: 24.07%
test 크기: (1400, 8), 이탈 비율: 24.07%
Pipeline 단계: ['preprocessor', 'model']
교차검증 최적 파라미터: {'model__max_depth': 6, 'model__min_samples_leaf': 20}
train 5-fold 평균 ROC-AUC: 0.679
학습 완료: 모델 선택과 전처리는 train 데이터 안에서만 수행했습니다.

-- STEP 7: 모델 평가 및 저장 --
ROC-AUC: 0.673
train OOF 선택 임계값: 0.269 (정밀도=0.365, 재현율=0.600)

[분류 리포트]

```

	precision	recall	f1-score	support
전 류	0.832	0.667	0.740	1063
이 탈	0.354	0.576	0.438	337
accuracy			0.645	1400
macro avg	0.593	0.621	0.589	1400
weighted avg	0.717	0.645	0.668	1400

기본 RandomForest는 트리 깊이에 제한이 없고 리프 최소 표본 수가 1이어서 train ROC-AUC가 1.000, test ROC-AUC가 0.635로 과적합을 보였다. Test 데이터 누수를 방지하기 위해 train 데이터 내부에서만 5-fold GridSearchCV를 수행했으며, max_depth=6, min_samples_leaf=20을 선택하였다. 그 결과 test ROC-AUC가 0.635에서 0.673으로 향상되었다.

기본 임계값 0.5에서는 이탈 재현율이 낮아, train out-of-fold 예측에서 목표 재현율 0.60을 만족하는 임계값 0.269를 선택하였다. 독립 test 데이터의 이탈 재현율은 0.576, F1-score는 0.438로 향상되었다. 임계값을 낮추면서 정확도는 감소했지만 더 많은 이탈 고객을 탐지하는 방향으로 모델의 운영 목적을 조정하였다.

종합실습 3 - 분석 자동화 · 리포트 생성

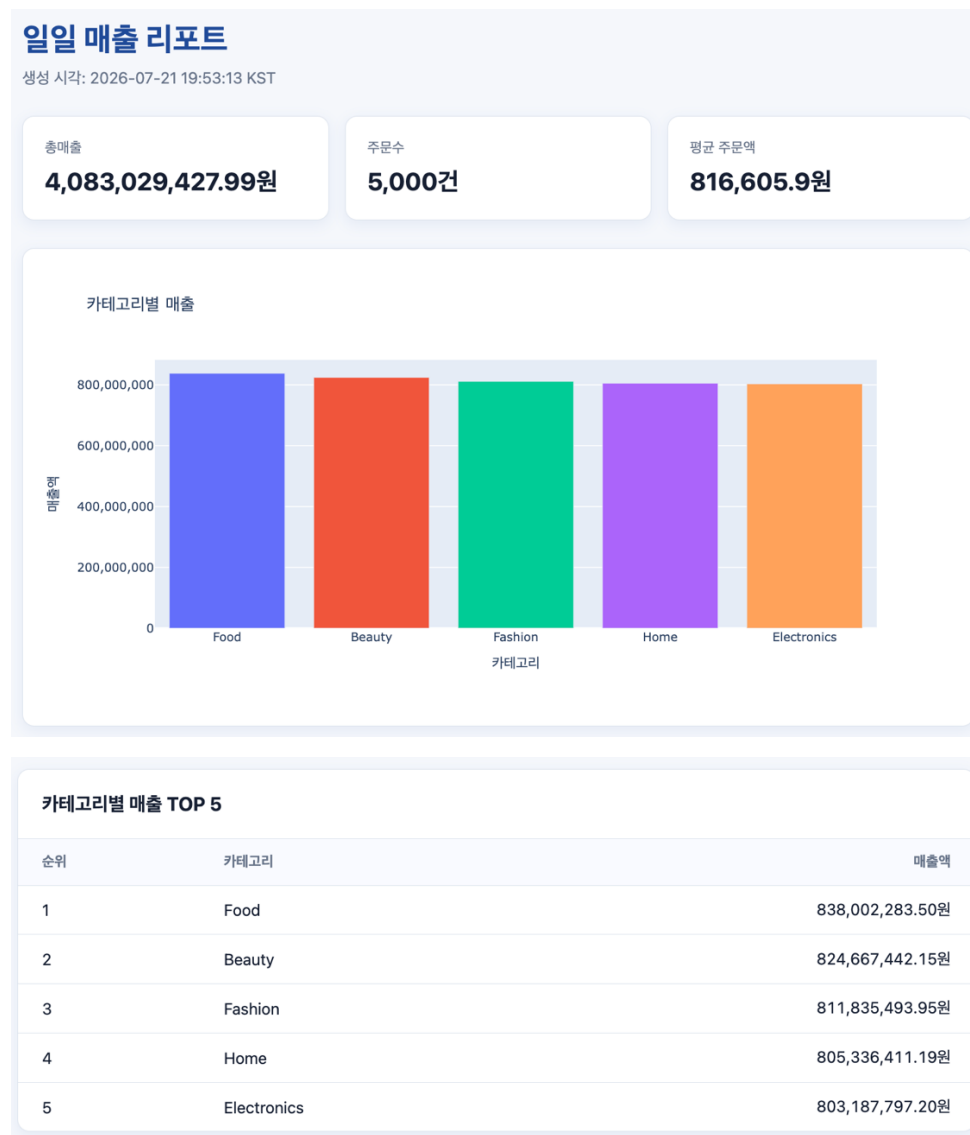
report.py

```
(.venv) limhaean@limhaean-ui-MacBookPro skala_python % python Python_data_0720_Day2/Total3/report.py
리포트 생성 완료: /Users/limhaean/workspace/skala_python/Python_data_0720_Day2/Total3/output/report_20260721_195313.html
```

report.py 실행 결과 타임스탬프가 포함된 HTML 리포트가 output 폴더에 생성되었다.

판매 데이터를 정제·집계하여 실행 시각이 포함된 HTML 리포트를 생성하였다.

output/report_20260721_193302.html (가장 최근 리포트)



Jinja2 HTML 리포트에 핵심 KPI와 카테고리별 매출 차트 및 순위표가 정상적으로 렌더링되었다.

```
python Python_data_0720_Day2/Total3/run_scheduler.py --interval 60
```

```
(.venv) limhaean@limhaean-ui-MacBookPro skala_python % python Python_data_0720_Day2/Total3/run_scheduler.py --interval 60
반복 실행 시작: 60초 간격 (종료: Ctrl+C)
리포트 생성 완료: /Users/limhaean/workspace/skala_python/Python_data_0720_Day2/Total3/output/report_20260721_195615.html
리포트 생성 완료: /Users/limhaean/workspace/skala_python/Python_data_0720_Day2/Total3/output/report_20260721_195715.html
리포트 생성 완료: /Users/limhaean/workspace/skala_python/Python_data_0720_Day2/Total3/output/report_20260721_195815.html
^C
사용자 요청으로 반복 실행을 종료했습니다.
```

--interval 60 옵션을 적용하여 60초마다 새로운 타임스탬프 리포트가 생성되는 것을 확인하였다.

```
pytest -v Python_data_0720_Day2/Total3/test_total3.py
```

```
(.venv) limhaean@limhaean-ui-MacBookPro skala_python % pytest -v Python_data_0720_Day2/Total3/test_total3.py
===== test session starts =====
platform darwin -- Python 3.11.15, pytest-8.4.2, pluggy-1.6.0 -- /Users/limhaean/workspace/skala_python/.venv/bin/python
cachedir: .pytest_cache
rootdir: /Users/limhaean/workspace/skala_python
collected 10 items

Python_data_0720_Day2/Total3/test_total3.py::test_aggregate_is_pure_and_returns_required_kpis PASSED [ 10%]
Python_data_0720_Day2/Total3/test_total3.py::test_render_report_creates_timestamp.html PASSED [ 20%]
Python_data_0720_Day2/Total3/test_total3.py::test_plotly_chart_is_embeddable PASSED [ 30%]
Python_data_0720_Day2/Total3/test_total3.py::test_run_once_creates_report_with_shared_pipeline PASSED [ 40%]
Python_data_0720_Day2/Total3/test_total3.py::test_report_and_scheduler_share_same_run_once PASSED [ 50%]
Python_data_0720_Day2/Total3/test_total3.py::test_retry_uses_exponential_backoff PASSED [ 60%]
Python_data_0720_Day2/Total3/test_total3.py::test_slack_notification_posts_report_link PASSED [ 70%]
Python_data_0720_Day2/Total3/test_total3.py::test_interval_loop_uses_shared_job PASSED [ 80%]
Python_data_0720_Day2/Total3/test_total3.py::test_schedule_uses_shared_job_and_clears_jobs PASSED [ 90%]
Python_data_0720_Day2/Total3/test_total3.py::test_main_routes_all_modes_to_the_shared_execution_functions PASSED [100%]

===== 10 passed in 0.58s =====
```

직접 실행 loop·schedule·cron 방식이 모두 동일한 run_once() 분석 파이프라인을 사용함을 테스트로 확인하였다.

Plotly 임베드



카테고리별 매출 데이터를 Plotly 막대 차트로 변환하고 Jinja2 HTML 리포트 내부에 임베드하였다.

이메일/슬랙 알림, 실패 재시도

pytest -v -s Python_data_0720_Day2/Total3/test_total3.py -k "plotly or retry or slack"

```
(.venv) limhaean@imhaean-ui-MacBookPro skala_python % pytest -v -s Python_data_0720_Day2/Total3/test_total3.py \
-k "plotly or retry or slack"
===== test session starts =====
platform darwin -- Python 3.11.15, pytest-8.4.2, pluggy-1.6.0 -- /Users/limhaean/workspace/skala_python/.venv/bin/python
cachedir: .pytest_cache
rootdir: /Users/limhaean/workspace/skala_python
collected 10 items / 7 deselected / 3 selected

Python_data_0720_Day2/Total3/test_total3.py::test_plotly_chart_is_embeddable PASSED
Python_data_0720_Day2/Total3/test_total3.py::test_retry_uses_exponential_backoff 리포트 생성 실패 (1/3), 1초 후 재시도합니다.
리포트 생성 실패 (2/3), 2초 후 재시도합니다.
PASSED
Python_data_0720_Day2/Total3/test_total3.py::test_slack_notification_posts_report_link PASSED

===== 3 passed, 7 deselected in 0.48s =====
(.venv) limhaean@imhaean-ui-MacBookPro skala_python %
```

slack 알림은 실제 외부 전송 대신 모의 웹훅으로 요청 데이터와 리포트 링크를 검증하였다. 데이터 로딩 실패 상황에서는 1초, 2초의 지수 백오프 후 재시도하도록 구현하였다.